

COMP3242/6242: Deep Learning

Week 5 — Object Detection and Image Segmentation

Stephen Gould
`stephen.gould@anu.edu.au`

Australian National University

Semester 1, 2026

Overview

- ▶ classification, localization and detection
 - ▶ intersection-over-union
- ▶ determining bounding boxes
 - ▶ sliding window detection and non-maximal suppression
 - ▶ object proposals and bounding box regression
- ▶ popular network architectures
 - ▶ R-CNN, Fast R-CNN and Faster R-CNN
 - ▶ SSD and YOLO
- ▶ segmentation tasks
 - ▶ superpixels, semantic, instance
- ▶ segmentation architectures
 - ▶ fully convolutional networks
 - ▶ upsampling and transposed convolutions
 - ▶ U-Net architecture
- ▶ models: mask R-CNN and segment anything (SAM)

Classification, Localization and Detection

- ▶ **classification** works great for images of single objects (or tagging scene type)
- ▶ ... but most scenes contain more than one object
- ▶ **object detection** is the task of finding multiple objects in an image by placing a bounding box around each object and assigning a label to that bounding box
- ▶ even more detailed localization — **instance segmentation**

Classification, Localization and Detection

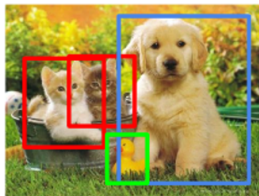
- ▶ **classification** works great for images of single objects (or tagging scene type)
- ▶ ... but most scenes contain more than one object
- ▶ **object detection** is the task of finding multiple objects in an image by placing a bounding box around each object and assigning a label to that bounding box
- ▶ even more detailed localization — **instance segmentation**



image classification



object localization



object detection



instance segmentation

Scoring Classification and Localization

- ▶ **classification:** K classes
 - ▶ **input:** image
 - ▶ **output:** class label
 - ▶ **evaluation metric:** accuracy
- ▶ **localization:** $([1, W] \times [1, H])^2$
 - ▶ **input:** image
 - ▶ **output:** box coordinates^{*ab*}
 - ▶ **evaluation metric:** intersection-over-union

^{*a*}usually normalized to $([0, 1] \times [0, 1])^2$

^{*b*}either (x, y, w, h) or (x_1, y_1, x_2, y_2)

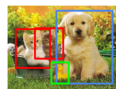
- ▶ **detection:** do both



→ "cat"



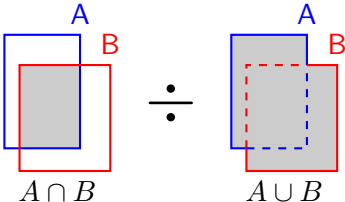
→ (x, y, w, h)



→ $\left\{ \begin{array}{l} \text{"cat"}, (x_1, y_1, w_1, h_1) \\ \vdots \\ \text{"dog"}, (x_n, y_n, w_n, h_n) \end{array} \right\}$

Intersection-over-Union (IoU)

- ▶ intersection-over-union (or Jaccard index) measures the similarity of two sets
- ▶ used in object detection to determine whether two bounding boxes are close

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$


- ▶ for a detection to be considered positive it must both
 - ▶ predict the correct class label, and
 - ▶ have IoU greater than a given threshold, denoted IoU_t
- ▶ typical thresholds are 0.3, 0.5 and 0.7 ($\text{IoU}_{0.3}$, $\text{IoU}_{0.5}$ and $\text{IoU}_{0.7}$)

Obtaining Bounding Boxes: Sliding Windows

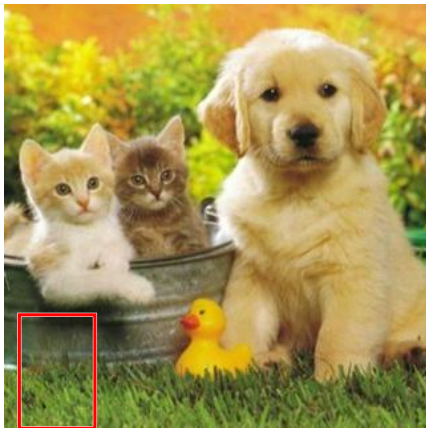


Obtaining Bounding Boxes: Sliding Windows



- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)

Obtaining Bounding Boxes: Sliding Windows



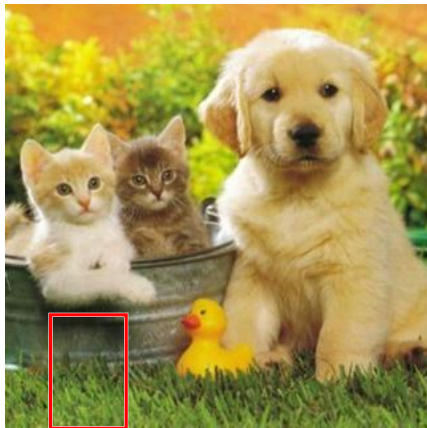
- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)
- ▶ slide bbox to next location

Obtaining Bounding Boxes: Sliding Windows



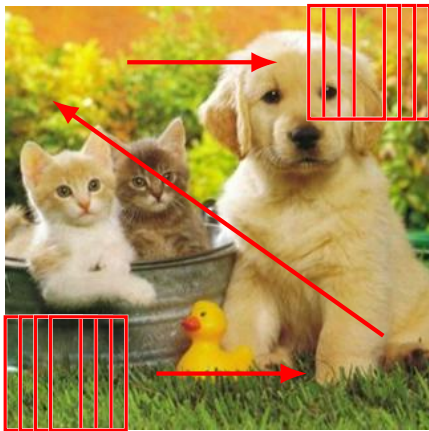
- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)
- ▶ slide bbox to next location
- ▶ ... and repeat

Obtaining Bounding Boxes: Sliding Windows



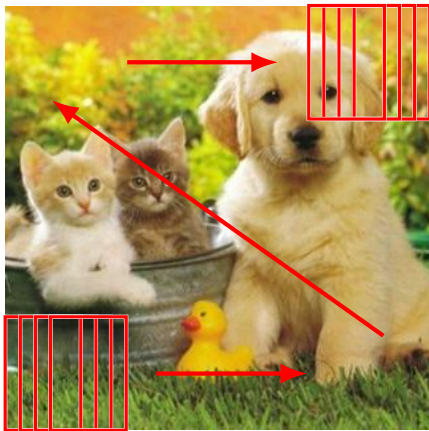
- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)
- ▶ slide bbox to next location
- ▶ ... and repeat

Obtaining Bounding Boxes: Sliding Windows



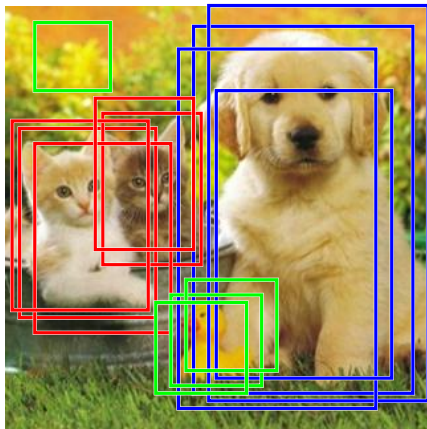
- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)
- ▶ slide bbox to next location
- ▶ ... and repeat

Obtaining Bounding Boxes: Sliding Windows



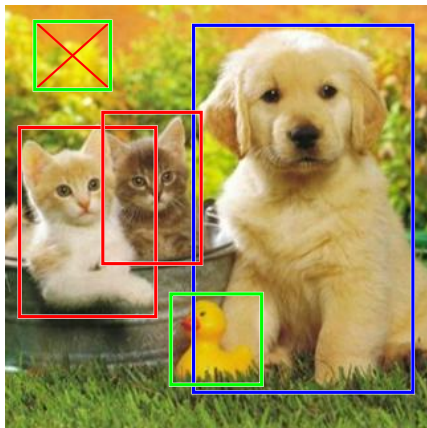
- ▶ pick bounding box size
- ▶ place bbox at corner of image
- ▶ run classifier on bbox contents
 - ▶ augmented with “background” category (i.e., $(K + 1)$ -way classification)
- ▶ slide bbox to next location
- ▶ ... and repeat
- ▶ increase bounding box size
- ▶ ... and repeat

Non-maximal Suppression (NMS)



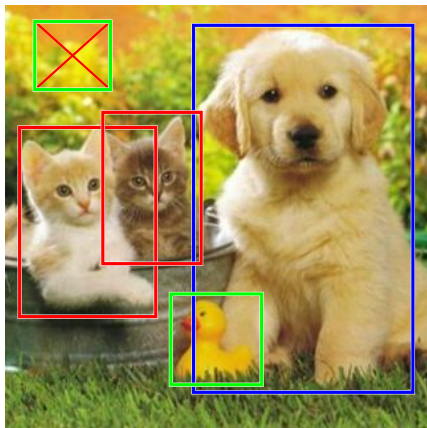
- ▶ at the end we're left with a set of bounding boxes
 - ▶ each with an associated probability distribution over object classes
 - ▶ many high-probability overlapping detections of the same object

Non-maximal Suppression (NMS)



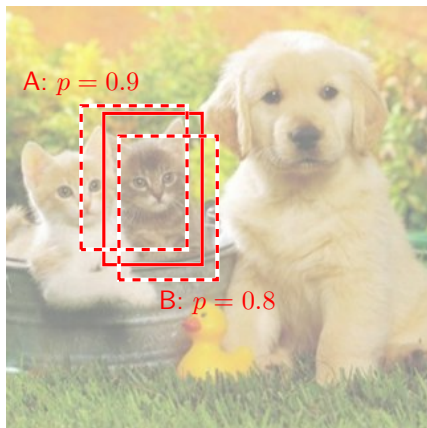
- ▶ at the end we're left with a set of bounding boxes
 - ▶ each with an associated probability distribution over object classes
 - ▶ many high-probability overlapping detections of the same object
- ▶ filter out low probability boxes
- ▶ apply non-maximal suppression to get rid of multiple detections of the same object

Non-maximal Suppression (NMS)



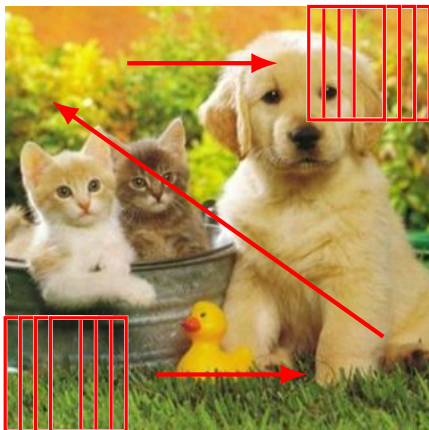
- ▶ at the end we're left with a set of bounding boxes
 - ▶ each with an associated probability distribution over object classes
 - ▶ many high-probability overlapping detections of the same object
- ▶ filter out low probability boxes
- ▶ apply non-maximal suppression to get rid of multiple detections of the same object
- ▶ false positives (FP) may remain
- ▶ missed detections cannot be recovered
- ▶ evaluate using mAP based on IoU_t
 - ▶ repeated detections count as FP

Duplicate Detections Example



- ▶ ground-truth bounding box (solid) provided by human annotator
- ▶ two bounding boxes, A and B , may still detect the same object
 - ▶ IoU below threshold for non-maximal suppression, so both kept
 - ▶ but IoU of each with ground-truth above threshold
- ▶ detection A will be counted as positive since it has a higher probability than B
- ▶ detection B will be counted as a negative since ground-truth already matched by a higher probability detection

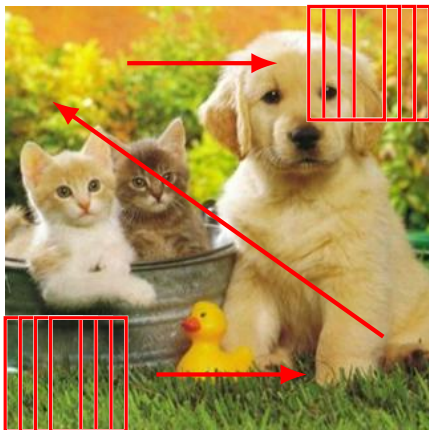
Counting Classifier Evaluations



- ▶ for a $H \times W$ image,
 - ▶ box bottom left can be anywhere from $x = 0$ to $W - 1$ and $y = 0$ to $H - 1$
 - ▶ with width and height anywhere from $w = 1$ to $W - x$ and $h = 1$ to $H - y$
 - ▶ therefore we have

$$\begin{aligned}\# \text{ boxes} &= \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} (W-x)(H-y) \\ &= \frac{W(W+1)}{2} \frac{H(H+1)}{2} \\ &\approx \frac{1}{4} (\# \text{ pixels})^2\end{aligned}$$

Counting Classifier Evaluations



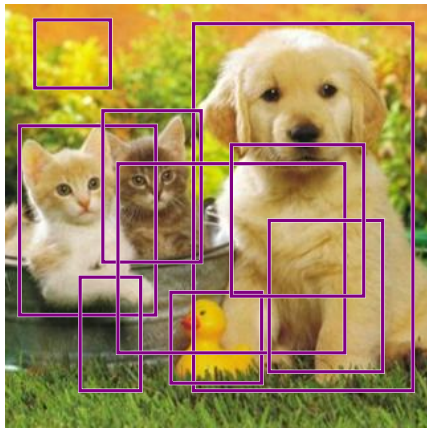
- ▶ for a $H \times W$ image,
 - ▶ box bottom left can be anywhere from $x = 0$ to $W - 1$ and $y = 0$ to $H - 1$
 - ▶ with width and height anywhere from $w = 1$ to $W - x$ and $h = 1$ to $H - y$
 - ▶ therefore we have

$$\begin{aligned}\# \text{ boxes} &= \sum_{x=0}^{W-1} \sum_{y=0}^{H-1} (W - x)(H - y) \\ &= \frac{W(W + 1)}{2} \frac{H(H + 1)}{2} \\ &\approx \frac{1}{4} (\# \text{ pixels})^2\end{aligned}$$

- ▶ e.g., 1280×720 image has 212G boxes!

Obtaining Bounding Boxes: Region Proposals

(Uijlings et al., "Selective Search", 2013)



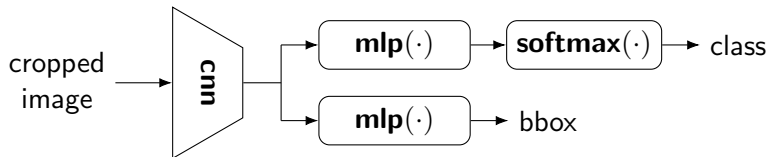
- ▶ find a small set of candidate boxes that are likely to contain objects
- ▶ often based on low-level image features (e.g., edges)
- ▶ "Selective Search" is a famous approach
 - ▶ combines bottom up over-segmentation with agglomerative clustering
 - ▶ returns 2,000 class-agnostic bounding box proposals per image

Classifying and Refining Bounding Boxes

- ▶ image cropped to bbox, resized, and fed into CNN classifier
- ▶ bounding box may be improved (tightened) by localizing

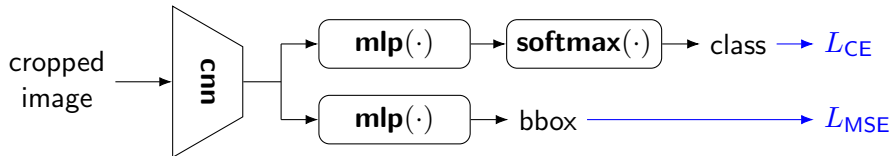
Classifying and Refining Bounding Boxes

- ▶ image cropped to bbox, resized, and fed into CNN classifier
- ▶ bounding box may be improved (tightened) by localizing



Classifying and Refining Bounding Boxes

- ▶ image cropped to bbox, resized, and fed into CNN classifier
- ▶ bounding box may be improved (tightened) by localizing



Training (with shared backbone).

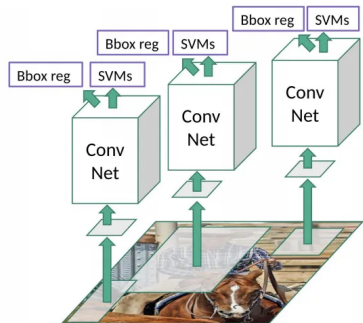
- ▶ classifier is trained with cross-entropy (CE) loss on true label, $y^{(i)}$
- ▶ bounding box regressor trained with MSE loss on true bounding box, $b^{(i)}$
 - ▶ only applied to non-background boxes (determined by IoU with ground-truth)
- ▶ total loss is a weighted combination of CE and MSE

$$\sum_{i=1}^N -\log p\left(y^{(i)} \mid x^{(i)}; \theta\right) + \frac{\lambda}{2} \left\| \text{bbox}(x^{(i)}; \theta) - b^{(i)} \right\|^2$$

R-CNN: Region-based CNN

(Girshick et al., 2014)

- ▶ CNN backbone pre-trained on ImageNet
- ▶ image crops resized to 227-by-227 (independent of aspect ratio)
- ▶ CNN backbone applied to image crops
- ▶ results in 4096-dim feature vector (per crop)
- ▶ linear classifier and bbox regression branches



- ▶ bbox branch predicts transformation $(\Delta x, \Delta y, \Delta w, \Delta h)$ of proposed region

$$\hat{b} = (x + w\Delta x, y + h\Delta y, w \exp \Delta w, h \exp \Delta h)$$

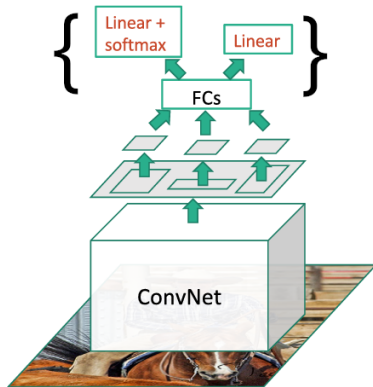
- ▶ translation proportional to box size
- ▶ scale in log-space

Fast and Faster R-CNN

(Girshick et al., 2015; Ren et al., 2015)

Fast R-CNN.

- ▶ introduces **RoI pooling** to avoid repeated evaluation of image crops
 - ▶ CNN backbone applied to full image
 - ▶ box proposals are projected onto feature map
 - ▶ pooling performed per projected box proposal for the region's feature vector representation
 - ▶ classification and bbox regression as before



Fast and Faster R-CNN

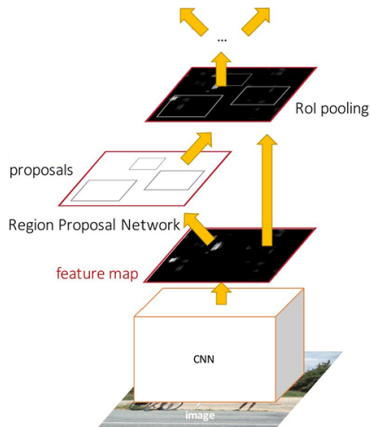
(Girshick et al., 2015; Ren et al., 2015)

Fast R-CNN.

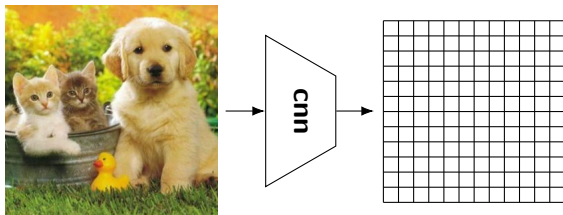
- ▶ introduces **RoI pooling** to avoid repeated evaluation of image crops
 - ▶ CNN backbone applied to full image
 - ▶ box proposals are projected onto feature map
 - ▶ pooling performed per projected box proposal for the region's feature vector representation
 - ▶ classification and bbox regression as before

Faster R-CNN.

- ▶ incorporate **region proposal network (RPN)** within CNN instead of Selective Search
 - ▶ predict and score bbox proposals
 - ▶ uses same CNN backbone!

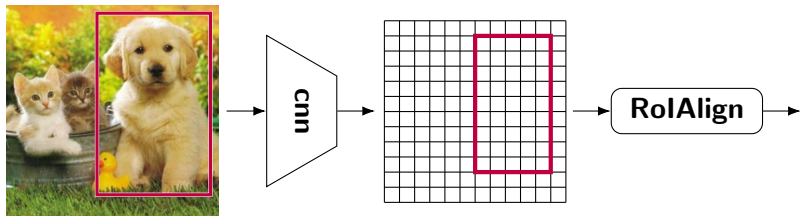


RoI Pooling Details



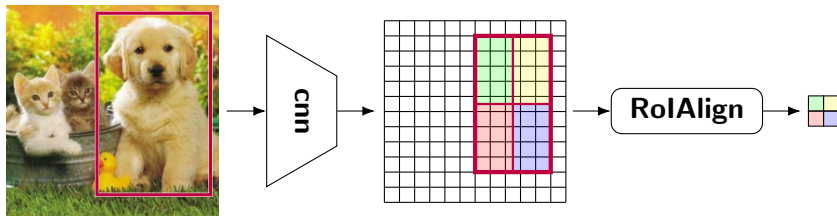
- CNN backbone produces feature map for entire image

RoI Pooling Details



- ▶ CNN backbone produces feature map for entire image
- ▶ proposal region projected to feature map

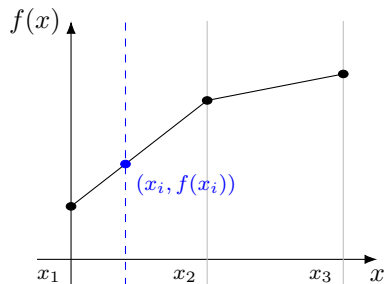
Rol Pooling Details



- ▶ CNN backbone produces feature map for entire image
- ▶ proposal region projected to feature map
- ▶ snap region to feature map grid for **RolPool**, or
- ▶ perform bilinear interpolation to align feature map to region for **RolAlign**
- ▶ apply max pooling over subregions (to all channels independently)
- ▶ produces a fixed size output feature map
 - ▶ e.g., 512-by-7-by-7 for Fast R-CNN

Linear Interpolation

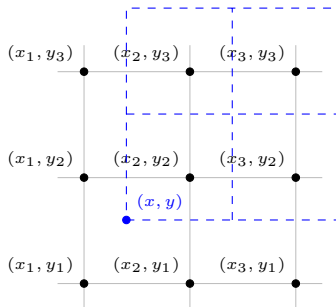
- ▶ let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a function defined on integer coordinates
- ▶ linear interpolation computes a piecewise linear approximation to the function



$$f^{\text{linear}}(x) = \left(\frac{x_2 - x}{x_2 - x_1} \right) f(x_1) + \left(\frac{x - x_1}{x_2 - x_1} \right) f(x_2)$$

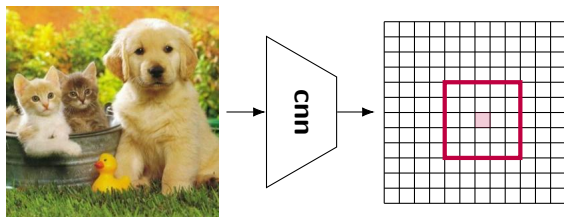
Bilinear Interpolation

- ▶ let $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ be a function defined on integer coordinates
- ▶ bilinear interpolation allows us to estimate the function at fractional coordinates
- ▶ computes weighted average of nearest neighbour points



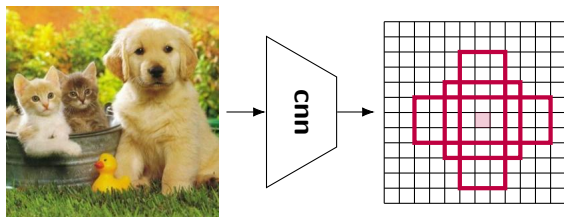
$$f^{\text{bilinear}}(x, y) = \frac{1}{(x_2 - x_1)(y_2 - y_1)} \begin{bmatrix} x_2 - x \\ x - x_1 \end{bmatrix}^T \begin{bmatrix} f(x_1, y_1) & f(x_1, y_2) \\ f(x_2, y_1) & f(x_2, y_2) \end{bmatrix} \begin{bmatrix} y_2 - y \\ y - y_1 \end{bmatrix}$$

Region Proposal Network (RPN) Details



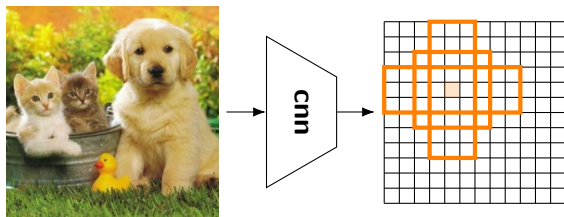
- associate **anchor boxes** with each location in feature map from backbone network

Region Proposal Network (RPN) Details



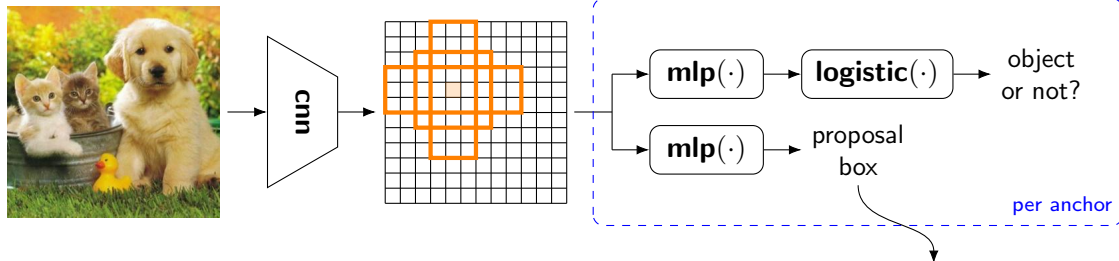
- associate **anchor boxes** with each location in feature map from backbone network

Region Proposal Network (RPN) Details



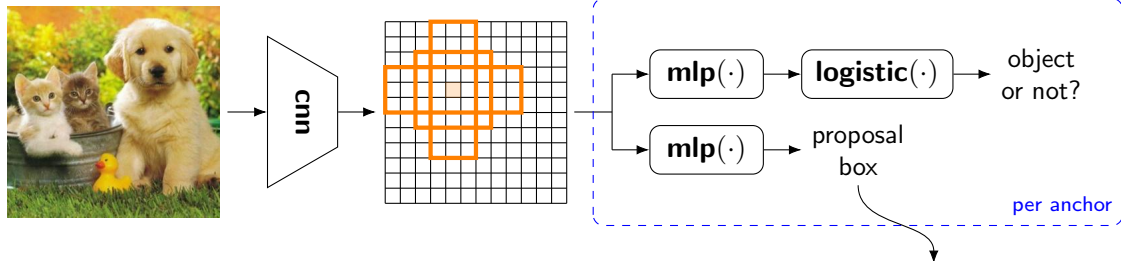
- associate **anchor boxes** with each location in feature map from backbone network

Region Proposal Network (RPN) Details



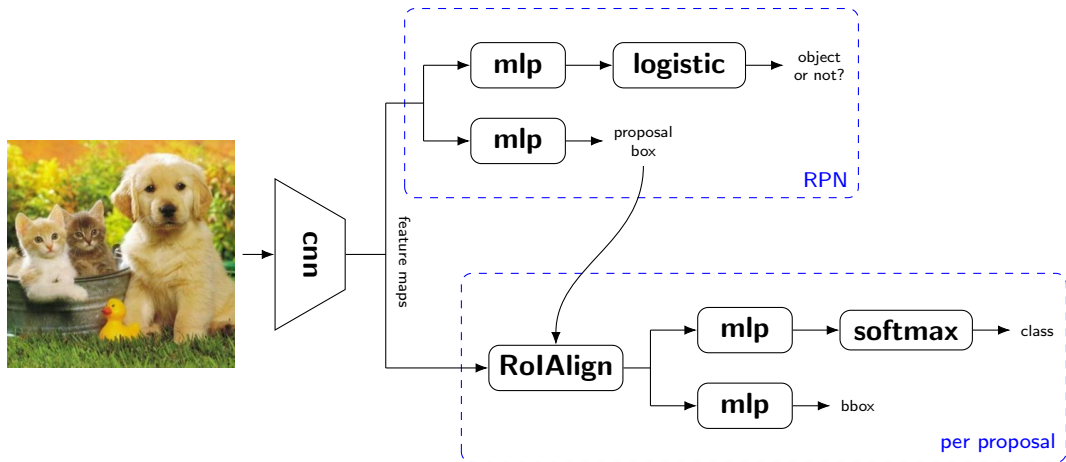
- ▶ associate **anchor boxes** with each location in feature map from backbone network
- ▶ classify each anchor box as object or not
- ▶ for positive boxes, also regress bounding box transformation

Region Proposal Network (RPN) Details

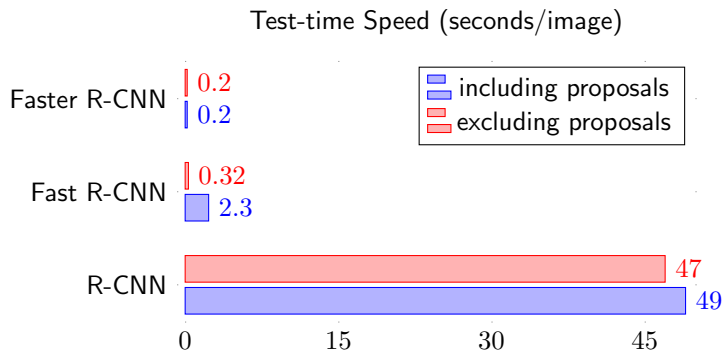


- ▶ associate **anchor boxes** with each location in feature map from backbone network
- ▶ classify each anchor box as object or not
- ▶ for positive boxes, also regress bounding box transformation
- ▶ use top- k scoring proposal boxes fed to RoIAlign (along with feature map)
- ▶ model is trained end-to-end using four loss components:
 - ▶ binary cross-entropy and transformation regression on anchor boxes
 - ▶ multi-class cross-entropy and bounding box regression on proposals

Faster R-CNN Architecture Summary

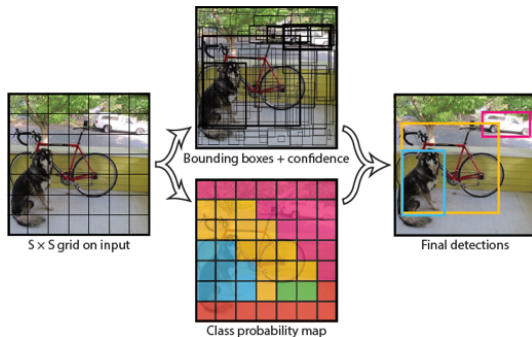


R-CNN, Fast R-CNN and Faster R-CNN Speed Comparison



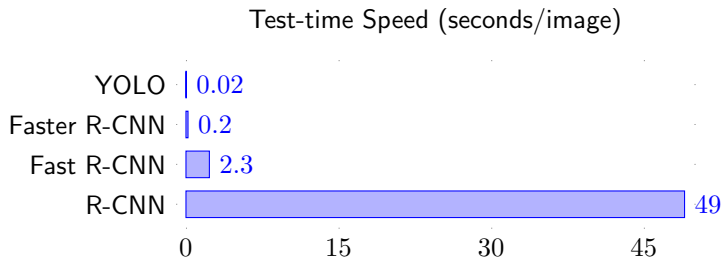
Single Stage Detection — You Only Look Once

(Redmon et al., “YOLO”, 2016; Liu et al., “SSD”, 2016)



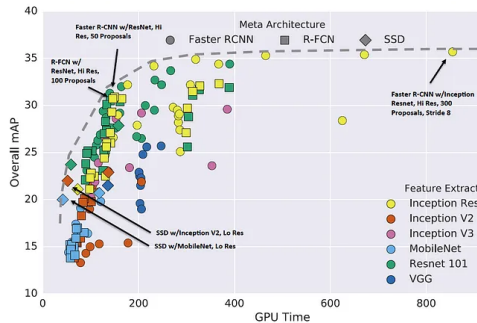
- ▶ divide image into $S \times S$ grid
- ▶ regress and classify m bounding boxes per grid location from feature maps
 - ▶ results in $S \times S \times (5m + K)$ tensor output
- ▶ apply thresholding and non-maximal suppression

YOLO Speed Comparison

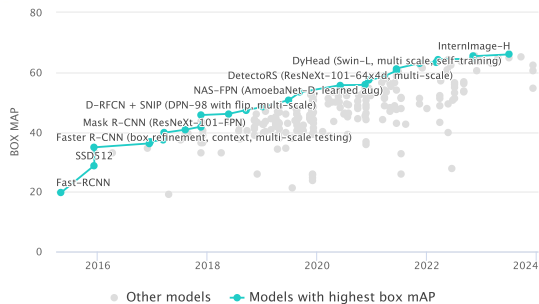


Object Detection Model Evaluation and Progress

- reporting speed is not enough, we also need to report accuracy (mAP)



(Huang et al., CVPR 2017)

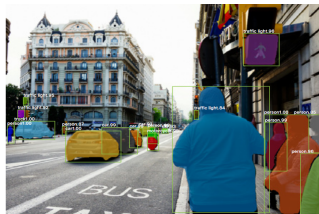
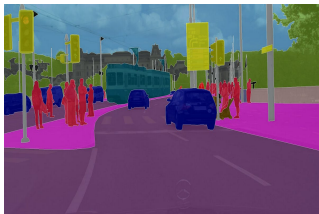
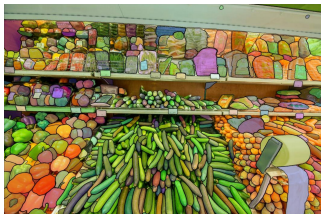


(paperswithcode.com/sota/object-detection-on-coco)

- performance depends on many factors: method, backbone architecture, image resolution, training regime, datasets, evaluation thresholds, etc.

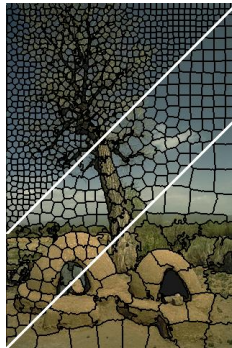
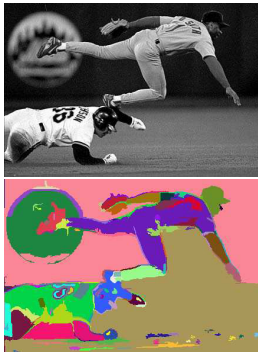
Segmentation Tasks

- ▶ image segmentation falls under the umbrella of **pixel labelling** and includes
 - ▶ **unsupervised segmentation**: break image up into meaningful regions or superpixels
 - ▶ typically classical (non-deep learning) techniques, e.g., Selective Search
 - ▶ **semantic segmentation**: label each pixel with as belonging to a given class
 - ▶ **instance segmentation**: label each pixel with an object/instance identifier



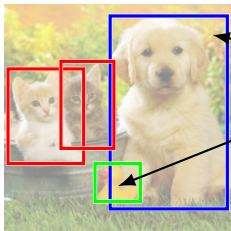
Superpixels

(Felzenszwalb and Huttenlocher, 2004; Achanta et al., 2010)

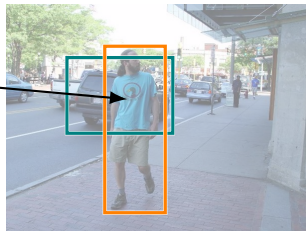


Semantic Segmentation

- **problem:** object detection leaves some pixels ambiguous

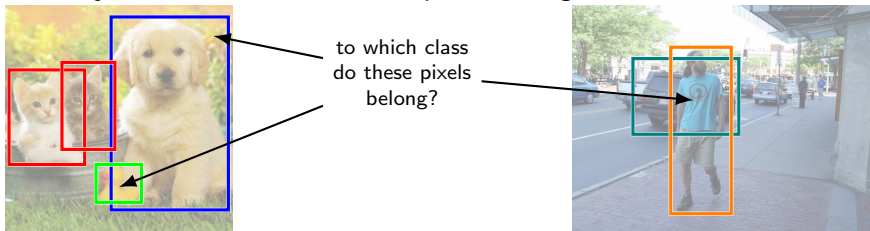


to which class
do these pixels
belong?

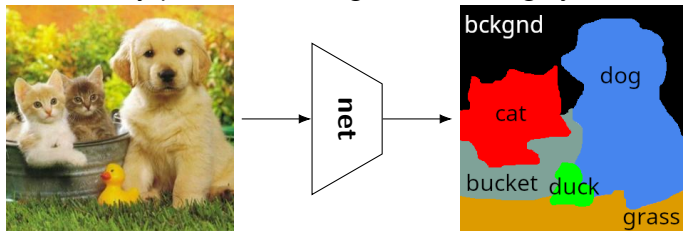


Semantic Segmentation

- **problem:** object detection leaves some pixels ambiguous

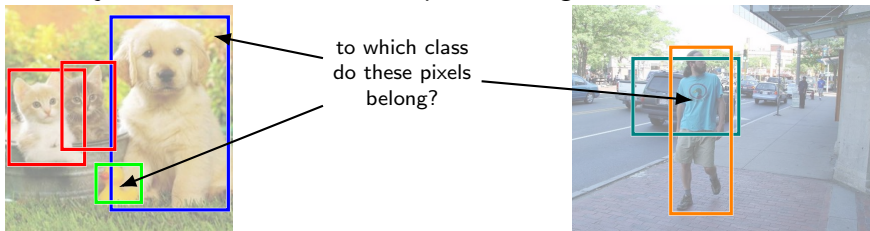


- **solution:** annotate every pixel in the image with a category label

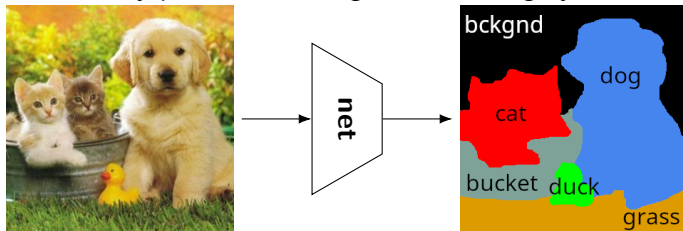


Semantic Segmentation

- **problem:** object detection leaves some pixels ambiguous



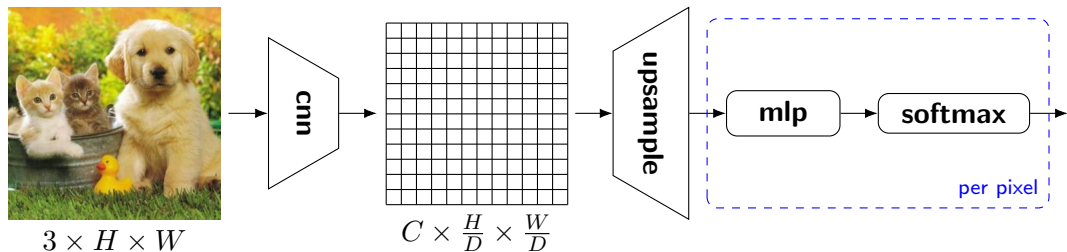
- **solution:** annotate every pixel in the image with a category label



- (does not distinguish between instances)

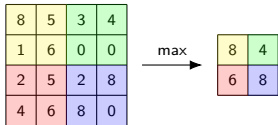
Fully Convolutional Network (FCN)

(Long et al., 2015)

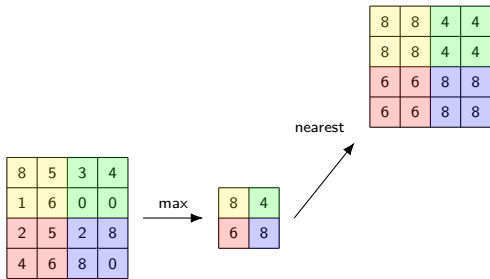


- ▶ pooling in the CNN backbone reduces the feature map size
- ▶ we need to upsample back to the original resolution for per-pixel labels
- ▶ the per-pixel MLP can be implemented using 1-by-1 convolutions

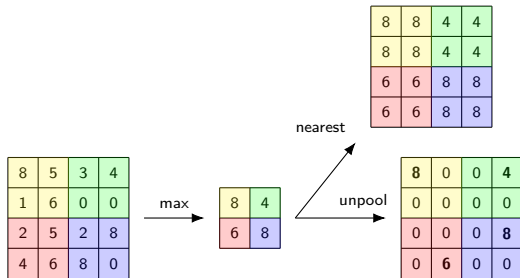
Upsampling



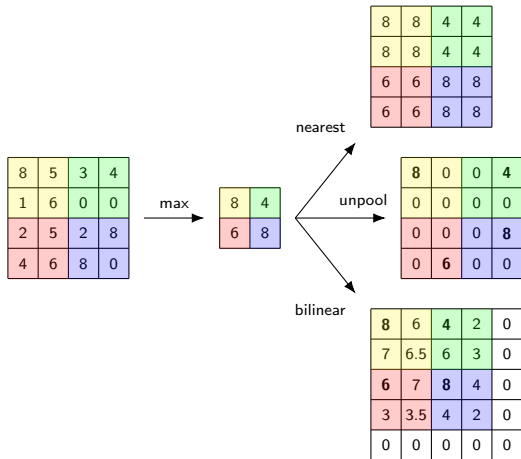
Upsampling



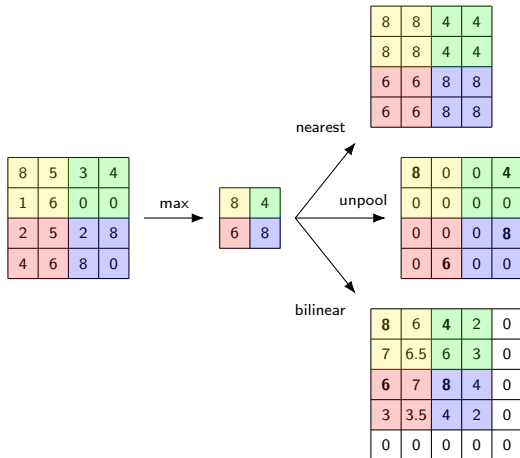
Upsampling



Upsampling

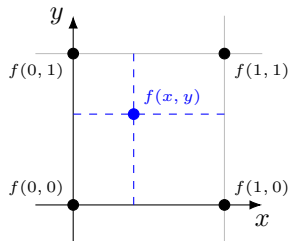


Upsampling



Bilinear interpolation.

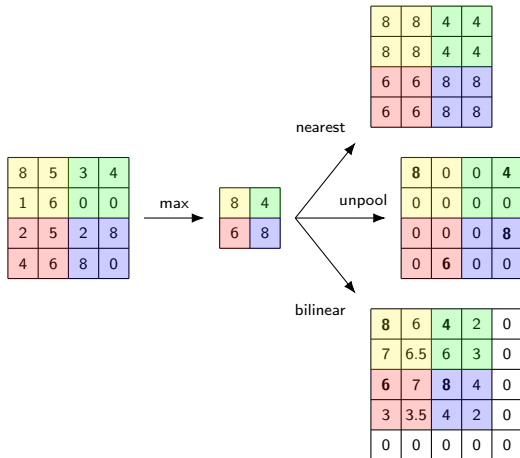
► consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$



$$f^{\text{nearest}}(x, y) = f_{\lfloor x \rfloor, \lfloor y \rfloor}$$

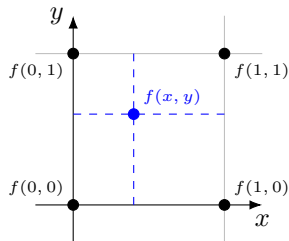
$$f^{\text{bilinear}}(x, y) = \begin{bmatrix} 1-x \\ x \end{bmatrix}^T \begin{bmatrix} f_{00} & f_{01} \\ f_{10} & f_{11} \end{bmatrix} \begin{bmatrix} 1-y \\ y \end{bmatrix}$$

Upsampling



Bilinear interpolation.

- consider a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$



$$f^{\text{nearest}}(x, y) = f_{\lfloor x \rfloor, \lfloor y \rfloor}$$

$$f^{\text{bilinear}}(x, y) = \begin{bmatrix} 1-x \\ x \end{bmatrix}^T \begin{bmatrix} f_{00} & f_{01} \\ f_{10} & f_{11} \end{bmatrix} \begin{bmatrix} 1-y \\ y \end{bmatrix}$$

- also bicubic interpolation (typical in image resizing) but requires a 4-by-4 grid

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8	7	4
filter, a :							
output, y :							

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	<table><tr><td>9</td><td></td><td>3</td><td></td><td>0</td><td></td><td>2</td><td></td><td>8</td><td></td><td>7</td><td></td><td>4</td></tr></table>	9		3		0		2		8		7		4
9		3		0		2		8		7		4		
filter, a :	<table><tr><td>1</td><td>-2</td><td>3</td></tr></table>	1	-2	3										
1	-2	3												
output, y :														

[†]not the mathematical inverse of a convolution

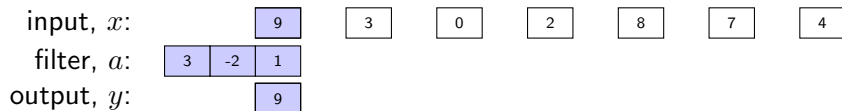
Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.



[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	<table><tr><td>9</td><td></td><td>3</td></tr></table>	9		3	<table><tr><td>0</td></tr></table>	0	<table><tr><td>2</td></tr></table>	2	<table><tr><td>8</td></tr></table>	8	<table><tr><td>7</td></tr></table>	7	<table><tr><td>4</td></tr></table>	4
9		3												
0														
2														
8														
7														
4														
filter, a :	<table><tr><td>3</td><td>-2</td><td>1</td></tr></table>	3	-2	1										
3	-2	1												
output, y :	<table><tr><td>9</td><td>-18</td></tr></table>	9	-18											
9	-18													

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	<table><tr><td>9</td><td></td><td>3</td></tr></table>	9		3	<table><tr><td>0</td></tr></table>	0	<table><tr><td>2</td></tr></table>	2	<table><tr><td>8</td></tr></table>	8	<table><tr><td>7</td></tr></table>	7	<table><tr><td>4</td></tr></table>	4
9		3												
0														
2														
8														
7														
4														
filter, a :	<table><tr><td>3</td><td>-2</td><td>1</td></tr></table>	3	-2	1										
3	-2	1												
output, y :	<table><tr><td>9</td><td>-18</td><td>30</td></tr></table>	9	-18	30										
9	-18	30												

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	<table><tr><td>9</td><td></td><td>3</td><td></td><td>0</td></tr></table>	9		3		0	<table><tr><td>2</td></tr></table>	2	<table><tr><td>8</td></tr></table>	8	<table><tr><td>7</td></tr></table>	7	<table><tr><td>4</td></tr></table>	4
9		3		0										
2														
8														
7														
4														
filter, a :	<table><tr><td></td><td>3</td><td>-2</td><td>1</td></tr></table>		3	-2	1									
	3	-2	1											
output, y :	<table><tr><td>9</td><td>-18</td><td>30</td><td>-6</td></tr></table>	9	-18	30	-6									
9	-18	30	-6											

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3		0	2	8	7	4
filter, a :		3	-2	1				
output, y :	9	-18	30	-6	9			

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3		0		2	8	7	4
filter, a :			3	-2	1				
output, y :	9	-18	30	-6	9	0			

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0		2	8	7	4
filter, a :			3	-2	1			
output, y :	9	-18	30	-6	9	0	2	

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, stride = 2). Reverse strided convolution.

input, x :	9	3	0		2		8	7	4
filter, a :				3	-2	1			
output, y :	9	-18	30	-6	9	0	2	-4	

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2		8	7	4	
filter, a :				3	-2	1			
output, y :	9	-18	30	-6	9	0	2	-4	14

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2		8		7	4	
filter, a :					3	-2	1			
output, y :	9	-18	30	-6	9	0	2	-4	14	-16

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8	7	4				
filter, a :					3	-2	1				
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8		7		4			
filter, a :							3	-2	1			
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31	-14

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8	7		4					
filter, a :						3	-2	1					
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31	-14	25

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8	7	4							
filter, a :							3	-2	1					
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31	-14	25	-8

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)[†]

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example ($a = (1, -2, 3)$, $\text{stride} = 2$). Reverse strided convolution.

input, x :	9	3	0	2	8	7	4								
filter, a :							3	-2	1						
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31	-14	25	-8	12

[†]not the mathematical inverse of a convolution

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4
filter, a :	1	-2	3				
previous, y :							
output, y :							

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4
filter, a :	1	-2	3				
previous, y :							
output, y :	9	-18	27				

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4
filter, a :		1	-2	3			
previous, y :	9	-18	27				
output, y :	9	-18	30	-6	9		

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4
filter, a :			1	-2	3		
previous, y :	9	-18	30	-6	9		
output, y :	9	-18	30	-6	9	0	0

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :

filter, a :

previous, y :

output, y :

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative ($a = (1, -2, 3)$, stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4				
filter, a :					1	-2	3				
previous, y :	9	-18	30	-6	9	0	2	-4	6		
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	24

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative ($a = (1, -2, 3)$, stride = 2). Scale, shift, sum.

input, x :	9	3	0	2	8	7	4						
filter, a :						1	-2	3					
previous, y :	9	-18	30	-6	9	0	2	-4	14	-16	24		
output, y :	9	-18	30	-6	9	0	2	-4	14	-16	31	-14	21

Learnable Upsampling — Transposed Convolution (aka Deconvolution)

Let $x \in \mathbb{R}^n$ be an input signal, $a \in \mathbb{R}^p$ be a filter kernel, and $s \geq 1$ be a stride. Mathematically, we can write the transposed convolution operator as

$$y_i = \sum_{j=1}^p a_j x_{\frac{i-j+s}{s}} \quad \text{for } i = 1, \dots, s(n-1) + p$$

where $x_{\frac{i-j+s}{s}}$ is defined to be zero if the index is fractional or greater than n .

Worked Example Alternative (a = (1, -2, 3), stride = 2). Scale, shift, sum.

input, x :

9

3

0

2

8

7

4

filter, a :

1

-2

3

previous, y :

9

-18

30

-6

9

0

2

-4

14

-16

31

-14

21

output, y :

9

-18

30

-6

9

0

2

-4

14

-16

31

-14

25

-8

12

Convolution and Transposed Convolution

Recall for convolution (with stride = 1) we have, in matrix form,

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Convolution and Transposed Convolution

Recall for convolution (with stride = 1) we have, in matrix form,

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

We can similarly write transposed convolution (with stride = 1) as

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n+p-1} \end{bmatrix} = \begin{bmatrix} a_1 & 0 & \dots & 0 \\ \vdots & a_1 & & \vdots \\ a_p & \vdots & \ddots & 0 \\ 0 & a_p & & a_1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Convolution and Transposed Convolution

Recall for convolution (with stride = 1) we have, in matrix form,

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-p+1} \end{bmatrix} = \begin{bmatrix} a_1 & \dots & a_p & 0 & \dots & 0 \\ 0 & a_1 & \dots & a_p & \dots & 0 \\ \vdots & & \ddots & & \ddots & \vdots \\ 0 & \dots & 0 & a_1 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

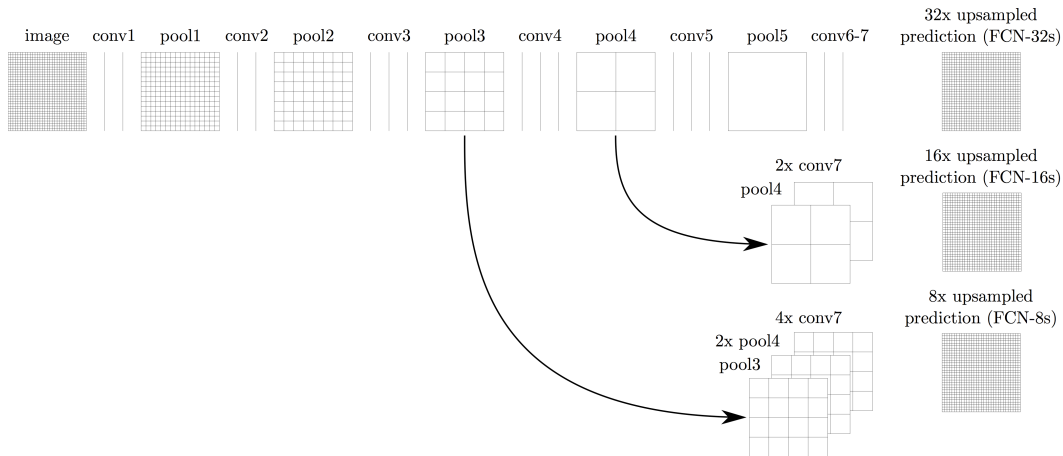
We can similarly write transposed convolution (with stride = 1) as

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n+p-1} \end{bmatrix} = \begin{bmatrix} a_1 & 0 & \dots & 0 \\ \vdots & a_1 & & \vdots \\ a_p & \vdots & \ddots & 0 \\ 0 & a_p & & a_1 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_p \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

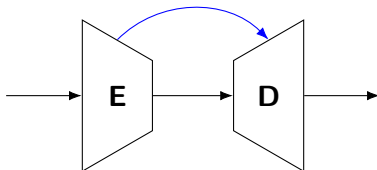
► so $y^{\text{conv}} = Ax$ and $y^{\text{trans}} = A^T x$, hence the name transposed convolution

Fully Convolutional Network (FCN) Architecture

(Long et al., 2015)



Skip Connections



- ▶ FCN predicts categories on upsampled feature maps
 - ▶ the resulting predictions are then fused via pixelwise summation
- ▶ a better approach (U-Net) combines feature maps from the downsampling branch with the upsampling branch and makes predictions on a combined feature map
 - ▶ consider a sequence of downsampled and upsampled feature maps

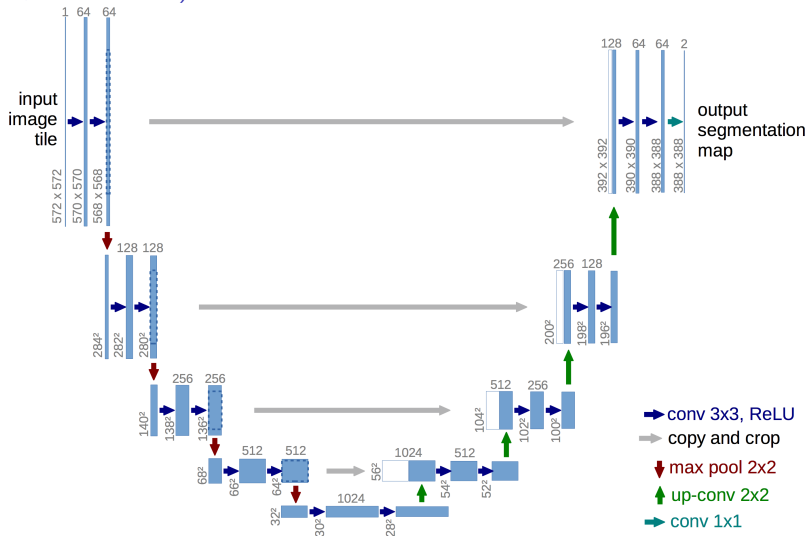
$$F_1^E \xrightarrow{\text{down}} F_2^E \xrightarrow{\text{down}} \dots \xrightarrow{\text{up}} F_2^D \xrightarrow{\text{up}} F_1^D$$

with $\text{shape}(F_\ell^E) = \text{shape}(F_\ell^D)$

- ▶ **skip connections** compute $F_{\ell-1}^D$ using $F_\ell^D \oplus F_\ell^E$ instead of just F_ℓ^D
- ▶ propagates contextual information from “encoder” to “decoder” layers

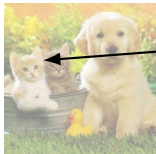
U-Net Architecture

(Ronneberger et al., MICCAI 2015)

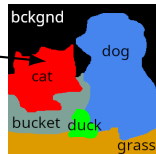


Instance/Panoptic Segmentation

- **problem:** semantic segmentation does not distinguish between instances

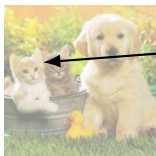


to which instance
does this pixel
belong?

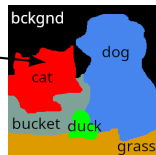


Instance/Panoptic Segmentation

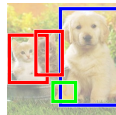
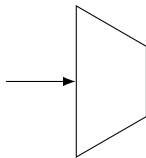
- **problem:** semantic segmentation does not distinguish between instances



to which instance
does this pixel
belong?

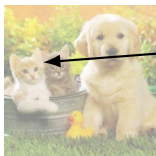


- **solution:** perform object detection and semantic segmentation jointly

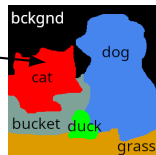


Instance/Panoptic Segmentation

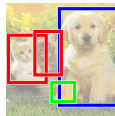
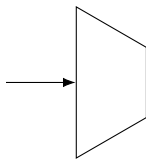
- **problem:** semantic segmentation does not distinguish between instances



to which instance
does this pixel
belong?



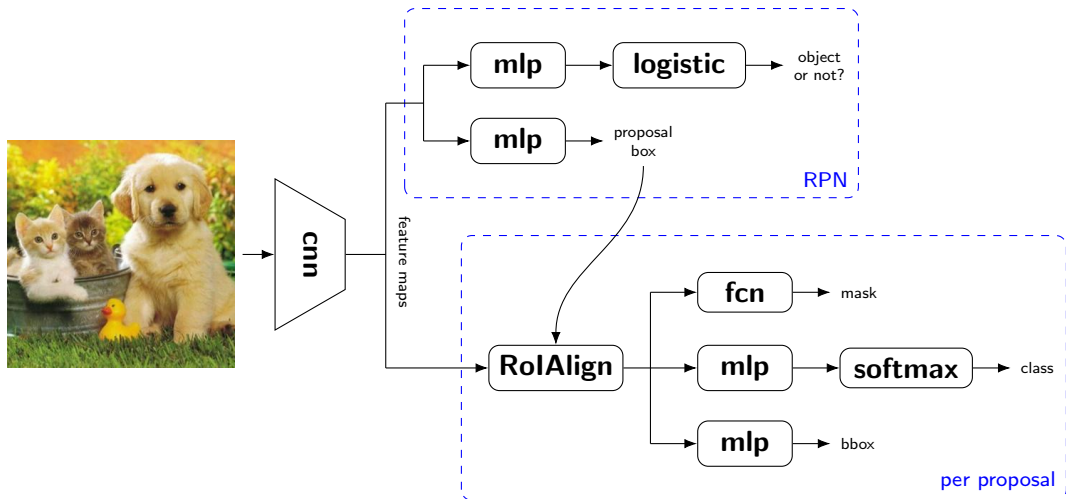
- **solution:** perform object detection and semantic segmentation jointly



- technically, we assign each pixel with a class label **and** instance identifier

Mask R-CNN

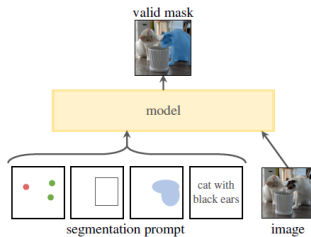
(He et al., 2017)



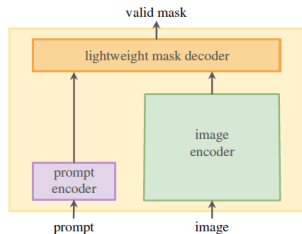
Segment Anything Model

(Kirillov et al., 2023)

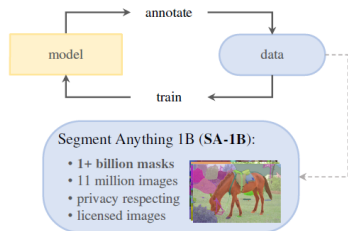
- ▶ **foundation model** based on ViT (later in the course) for segmentation
- ▶ trained on 1B pseudo-ground-truth masks and 11M images
 - ▶ bootstrapped from 4.3M human annotated masks and 120k images
 - ▶ followed by a further 5.9M masks and 180k images
- ▶ filtering process to eliminate poor pseudo-labels



(a) **Task:** promptable segmentation



(b) **Model:** Segment Anything Model (SAM)



(c) **Data:** data engine (top) & dataset (bottom)